

Bone Fracture Detection in X-rays

A learned CNN vs a wavelet Scattering Network, explained with six XAI methods

Visual Intelligence project

MSc in Artificial Intelligence • University of Verona • 2025/2026

The task and what we want to learn

Task. Binary classification of X-rays: **fractured** vs not fractured (all body regions).

Questions

1. Learned features (**CNN**) or fixed wavelets (**ScatNet**)? (+ a pretrained **ResNet18** as reference)
2. What do their first-layer filters look like?
3. Which XAI method explains them best, and which ones *cannot* be used on ScatNet?
4. Does our Occlusion (from scratch) match Captum's?

PyTorch • Kymatio • Captum • runs on a Kaggle GPU



Test X-rays: top row fractured, bottom row not fractured.

Dataset: Bone Fracture Multi-Region X-ray (Kaggle)

split	images
train	9,246
val	829
test	506

- grey X-rays of all body regions, very different sizes
- loaded in **grey**, 224×224
- **train**: CV + final training; **val**: picks the final epoch; **test**: used once
- augmentation: flip, $\pm 10^\circ$ rotation, shift, zoom, brightness, contrast

```
dataset_overview.png  
produced by the Kaggle run (./run.sh push, then ./run.sh  
get)
```

Studying the data: the same X-ray appears several times

Many images are rotated / flipped / re-cropped copies of one radiograph. We match 32×32 thumbnails ($\text{cosine} \geq 0.97$, 8 flips/rotations):

- – train images (–%) have a copy in train (– groups)
- – test images (–%) have a copy in train

Consequences

- **grouped** k-fold: copies never cross folds
- we also report accuracy on **clean** test images (no copy in train)

Copies rotated by other angles are missed: counts are a lower bound.

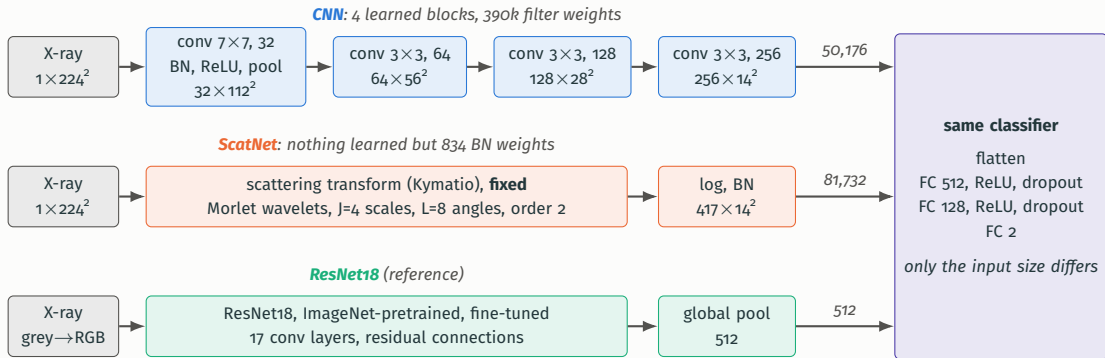
`duplicates_train.png`

produced by the Kaggle run (`./run.sh push`, then
`./run.sh get`)

`duplicates.tex`

produced by the Kaggle run (`./run.sh push`, then
`./run.sh get`)

Three feature extractors, one identical classifier



CNN and ScatNet both end on a **14×14 grid** (4 poolings $\leftrightarrow$ averaging at scale $2^4 = 16$ px). Parameters: CNN 26.1M • ScatNet 41.9M (all in the classifier) • ResNet18 11.5M.

ScatNet: a CNN whose filters are wavelets fixed in advance

Morlet wavelets $\psi_{j,\theta}$ (scale 2^j , angle θ), low-pass

ϕ_j :

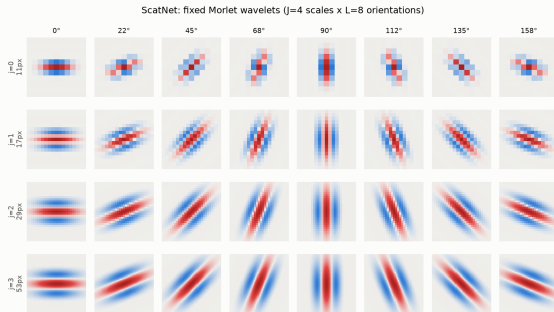
$$S_0 x = x \star \phi_j$$

$$S_1 x = |x \star \psi_{j_1, \theta_1}| \star \phi_j$$

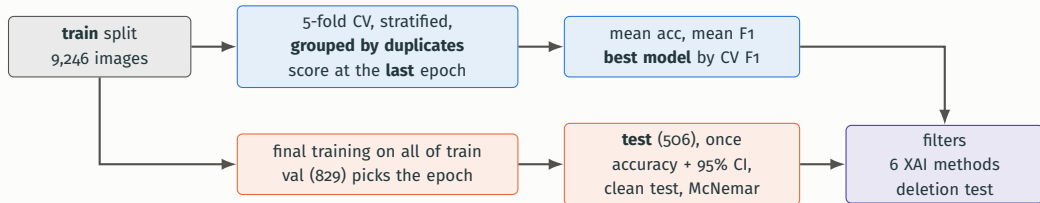
$$S_2 x = ||x \star \psi_{j_1, \theta_1}| \star \psi_{j_2, \theta_2}| \star \phi_j, j_2 > j_1$$

$$1 + JL + L^2 \frac{J(J-1)}{2} = 1 + 32 + 384 = \mathbf{417} \text{ maps of } \frac{224}{2^4} = 14^2.$$

- modulus = non-linearity, averaging = pooling
- invariant to shifts $< 2^j$, stable to small deformations
- **nothing to learn** (and nothing adapts to fractures)



Training and evaluation protocol



- AdamW, weight decay 10^{-4} , cosine schedule, 20 epochs, batch 32
- lr 10^{-3} (CNN, ScatNet), 10^{-4} (ResNet18 fine-tuning)
- cross-entropy; F1/recall for the **fractured** class
- nothing is selected on a validation fold $\Rightarrow$ CV is not optimistic
- the best model is chosen *without* the test set
- one command: `./run.sh push` (Kaggle GPU)

Cross-validation on the training set (mean $\pm$ std over 5 folds, %)

`cv.tex`

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

`learning_curves.png`

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

Test set (506 images, used once)

`test.tex`

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

`confusion_matrices.png`

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

`roc.png`

produced by the Kaggle run
(`./run.sh push`, then `./run.sh get`)

Which model is best, and why

model_comparison.png

produced by the Kaggle run (`./run.sh push,`
then `./run.sh get`)

McNemar (same test images): CNN vs ScatNet
 $p = \text{---}$, CNN vs ResNet18 $p = \text{---}$, ScatNet vs
ResNet18 $p = \text{---}$.

Best by CV F1: –

test accuracy CNN –%, ScatNet –%, ResNet18 –%
(target $\geq 75\%$)

- **ScatNet**: fixed, generic edge/texture energy at 4 scales $\times$ 8 angles; stable, but cannot adapt to thin, irregular fracture lines; its 42M weights all sit in the classifier
- **CNN**: filters and a hierarchy learned *for* X-rays, from only 9k images
- **ResNet18**: the same learned hierarchy, deeper, with residual connections and *pre-trained* on 1.2M images $\Rightarrow$ better features from little data, fewest parameters

Filters: learned (CNN) vs fixed (ScatNet)

`filters_cnn.png`

produced by the Kaggle run (`./run.sh push,`
then `./run.sh get`)

- CNN: 32 filters 7×7 learned with augmentation: oriented edges, blobs, contrast detectors
- ScatNet: Morlet wavelets, every scale and angle, whether useful or not

`frequency_coverage.png`

produced by the Kaggle run (`./run.sh push,`
then `./run.sh get`)

Power spectrum of the first layer: wavelets tile the frequency plane uniformly, the CNN concentrates where the X-rays need it.

Six XAI methods, and where they cannot be used

family	method	idea	CNN	ScatNet
gradient	Saliency	$ \partial f_c / \partial x $	✓	✓
gradient	Integrated Gradients	$(x - x_0) \int_0^1 \partial_x f_c(x_0 + \alpha(x - x_0)) d\alpha, x_0 = \text{black}$	✓	✓
modified gradient	Guided Backprop	only positive signals flow back through the ReLUs	✓	~ partly
class activation	Grad-CAM	last conv maps weighted by their mean gradient	✓	✗ no
perturbation	Occlusion	score drop when a 16×16 window turns black	✓	✓
surrogate	LIME	ridge model on 1000 random on/off 16×16 patches	✓	✓

- **Grad-CAM** needs a *learned* convolutional feature map: ScatNet's maps are fixed wavelet coefficients $\Rightarrow$ not applicable
- **Guided Backprop** modifies the ReLU backward pass: ScatNet's non-linearity is the complex modulus, so only the classifier is "guided"
- gradient- and perturbation-based methods only need $\partial f / \partial x$ or forward passes $\Rightarrow$ any model

XAI on the CNN (2 images per class, true class explained)

`xai_cnn.png`

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

`xai_scatnet.png`

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

Our Occlusion (from scratch) vs Captum's

Algorithm (Zeiler & Fergus 2014)

1. score $f_c(x)$ of the class
2. slide a 16×16 black window, stride 8
3. drop $f_c(x) - f_c(x_{\text{occluded}})$
4. each pixel: mean drop of the windows covering it

Plain PyTorch, batched

(src/occlusion_scratch.py).

Result: $\max |\text{ours} - \text{Captum}| = -$, Pearson $r \geq -$ on every image and model $\Rightarrow$ identical up to float error (also a unit test).

occlusion_scratch_vs_captum.png
produced by the Kaggle run (./run.sh push, then
./run.sh get)

Which method gives the best attributions?

deletion_curves.png

produced by the Kaggle run (`./run.sh push`, then
`./run.sh get`)

Deletion test: black out 16×16 patches, most important first; a faithful map makes the class probability fall fast (lower area = better; *Random* is the bar to beat).

deletion.tex

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

Most faithful: CNN –, ScatNet –.

Do the methods agree?

`xai_agreement.png`

produced by the Kaggle run (`./run.sh push`, then `./run.sh get`)

Spearman correlation of patch-level maps. Gradient methods agree with each other, perturbation methods with each other; the two families disagree more $\Rightarrow$ each answers a different question (local sensitivity vs effect of removing a region).

Discussion and conclusions

- **Models:** all above the 75% target; – is best by cross-validation and on the test set. Learned (and pre-trained) features beat fixed wavelets for thin, irregular fracture lines; ScatNet stays competitive with no learned filter at all.
- **Data:** near-duplicate X-rays inflate scores $\Rightarrow$ grouped CV + clean-test accuracy (CNN –, ScatNet –, ResNet18 –%).
- **Filters:** CNN filters look like oriented edge/blob detectors, related to but less uniform than Morlet wavelets.
- **XAI:** perturbation methods (Occlusion, LIME) are the most faithful but $10\text{--}100\times$ slower and coarse; gradients are sharp but noisy; Integrated Gradients is the best compromise. Grad-CAM does not apply to ScatNet; Guided Backprop only partly.
- **Our Occlusion** = Captum's up to floating-point error.
- **Limits:** rotated duplicates not detected; few explained images; one baseline (black) for all methods.

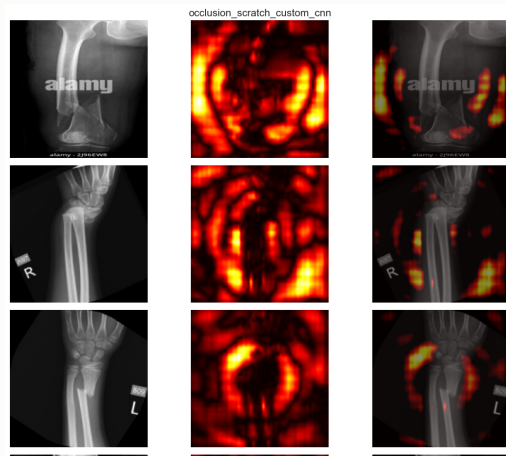
Backup: the previous iteration and what we changed

First version (team notebook, 10 epochs, RGB input):

model	CV acc.	test acc.
CNN	97.4	92.9
ResNet18	98.3	94.7
ScatNet	96.7	90.5

Fixed since

- CNN and ScatNet had *different* classifiers (exam!)
- CV folds mixed copies of one X-ray; fold score taken at the best epoch $\Rightarrow$ 4–6 pt gap CV/test
- grey baseline for Occlusion (right): highlights the black background
- grey images fed as 3 identical channels



v1 Occlusion with a grey baseline: the evidence sits on the background edges.

Backup: reproducibility and tests

- `notebooks/main.ipynb`: settings + calls to `src/` (data, models, training, xai, plots, report)
- Kaggle: `./run.sh push [-quick] → status → get → slides`; dataset attached through `kernel-metadata.json`
- `pytest`: data loading and caching, duplicate detection (planted flips/rotations/leaks), identical classifiers, every XAI method on every model, Occlusion scratch = Captum, Integrated Gradients completeness, deletion test ranks the right map first, grouped folds never split a group, a model overfits 16 images, the whole notebook on synthetic X-rays
- seeds fixed; every number in these slides is read from `results/summary.json`